

AI-Driven Website Summariser Web App

Complete Technical Solution, Architecture & Setup Documentation

Candidate Name: Priyanshu Kumar

Role Applied: Tech / Software Engineer (React / Angular)

GitHub Repository: github.com/priyanshujaiswalku/ai-web-site-summariser

Live Working Demo: ai-website-summariser-c2qn.onrender.com

1. Executive Summary & Problem Brief

This project is a production-grade full-stack web application designed and built for the **Indus Net Technologies Assignment 1** brief. The user provides any public website URL; the application fetches the target webpage, extracts and cleans readable textual content (stripping clutter, navigation, ads, and scripts), dispatches it to a high-speed AI language model (Groq LLM), and presents a concise, structured 4–6 sentence summary in an intuitive user interface.

2. End-to-End Architecture & Workflow

React (Vite) Frontend → Express REST API → Axios & Cheerio Scraper → Groq LLM API → Clean UI Result

Layer	Technologies Used	Core Responsibilities
Frontend	React 19, Vite, Modular CSS	URL validation, responsive UI, loading states, real-time error handling, clean presentation card.
Backend API	Node.js, Express, CORS, Dotenv	REST API routing, request validation, secure credential isolation, static serving of client bundle.
Scraping & Extraction	Axios, Cheerio	HTTP fetching with timeout, HTML parsing, script/nav/footer stripping, DOM text extraction.
AI Summarisation	Groq Chat Completions API (Llama / GPT models)	High-throughput contextual summarisation, structured prompt engineering, multi-model fallback.
Deployment	Render Cloud Platform	Single-tier unified deployment serving API and production React static bundle concurrently.

3. Key Engineering Highlights & Decisions

A. Secure Backend Proxy Architecture

Browser security policies (CORS) prevent browsers from scraping arbitrary third-party websites. Moreover, embedding AI API keys inside client-side JavaScript exposes secrets. The Express backend serves as a secure proxy: all external network requests and LLM invocations occur server-side, keeping credentials safe.

B. Intelligent Content Extraction with Cheerio

Raw webpage HTML contains large amounts of non-content noise (header navigation, footers, advertisement iframes, scripts, and stylesheets). The backend scraper strips `<script>`, `<style>`, `<nav>`, `<footer>`, and `<aside>` tags, then pulls readable text from semantic elements (`h1-h6`, `p`, `li`, `td`). If semantic elements are sparse, it gracefully falls back to structured body extraction.

C. High-Speed AI Engine & Resilient Fallback

Groq was selected for its ultra-low latency inference engine. The service includes automatic model fallback (prioritizing `openai/gpt-oss-20b`, `openai/gpt-oss-120b`, and Llama models) and a secondary remote fallback mechanism, ensuring zero downtime even if an individual AI model endpoint is temporarily unavailable.

D. Comprehensive Error Handling & Validation

Both client and server validate input URLs (ensuring valid HTTP/HTTPS protocol). The backend catches unreachable hosts, HTTP 403/404 errors, network timeouts, and insufficient extractable text, delivering clean, actionable error messages to the user interface.

4. Project Structure

```
ai-website-summariser/
├── client/                                # Frontend Application (React + Vite)
│   ├── src/
│   │   ├── api/summarize.js             # API communication layer
│   │   ├── components/                  # UrlForm, LoadingIndicator, SummaryCard, ErrorMessage
│   │   ├── App.jsx                      # Main state orchestration
│   │   └── App.css                      # Responsive styling
│   └── package.json
├── server/                               # Backend Application (Node.js + Express)
│   ├── routes/summarize.route.js        # POST /api/summarize endpoint
│   ├── services/
│   │   ├── scrape.service.js           # Cheerio HTML parser & text extractor
│   │   └── ai.service.js               # Groq API client with fallback logic
│   ├── middleware/logger.js            # Request logging middleware
│   ├── server.js                       # Express application entrypoint
│   └── package.json
└── README.md                           # Setup instructions and documentation
```

5. How to Run Locally

1. Clone Repository:

```
git clone https://github.com/priyanshujaiswalku/ai-website-summariser.git
cd ai-website-summariser
```

2. Start Backend Server:

```
cd server && npm install  
cp .env.example .env (Add your GROQ_API_KEY)  
npm start (Runs on http://localhost:5000)
```

3. Start Frontend (Development Mode):

```
cd ../client && npm install  
npm run dev (Runs on http://localhost:5173)
```

6. Verification & Demo Endpoints

- **Live Cloud Application:** <https://ai-website-summariser-c2qn.onrender.com>
- **Health Check Endpoint:** `GET /api/health` (Returns service status, version, and AI configuration)
- **Built-In Demo Article:** `GET /demo-article` (Self-contained test article for instant zero-dependency testing)

Submission prepared for Indus Net Technologies Ltd.

Candidate: Priyanshu Kumar